

UNIVERSIDAD DE COSTA RICA

Escuela de Ciencias de la Computación e Informática

Programación Paralela y Concurrente

Proyecto Semestral

Alineamiento de Secuencias de ADN

Implementación Paralela con Pthreads, MPI y OpenCL

Ciclo lectivo:

I Semestre 2026

Nivel:

IV Semestre — Ingeniería en C. de la Computación

Modalidad:

M.Sc. Sleyter Angulo

Lenguaje:

C / C++ con Pthreads, MPI y OpenCL

Nota: Este proyecto cuenta con tres entregables (semanas 8, 12 y 17). En TODOS los entregables el programa debe compilar y ejecutar sin errores. El código debe incluir casos de prueba.

PARTE I: Problema

Resumen

El problema elegido para este semestre implementa una solución de fuerza bruta para el alineamiento exacto de secuencias de ADN de múltiples patrones. El programa busca coincidencias exactas de múltiples cadenas de nucleótidos en una larga secuencia de ADN. La implementación secuencial está diseñada para ser clara y comprensible para los estudiantes, mientras ofrece muchas oportunidades de paralelización y optimización.

Esta tarea aborda conceptos clave que muchos estudiantes encuentran difíciles de aplicar en escenarios prácticos: condiciones de carrera (**race conditions**), reducciones, operaciones colectivas y comunicaciones punto a punto. También cubre el problema de la generación paralela de secuencias pseudoaleatorias y estrategias para notificar y detener cómputos especulativos cuando se encuentran coincidencias.

I. Idea y Contexto

a) Idea

La paralelización de programas requiere enfoques diferentes dependiendo del modelo de programación utilizado. Entender estas variaciones es esencial para que los estudiantes exploren técnicas avanzadas y aborden eficazmente los desafíos de la programación paralela en plataformas heterogéneas modernas.

b) El problema de la tarea

Las secuencias de ADN pueden representarse como cadenas de caracteres, donde cada carácter representa un tipo de nucleótido. El **problema de alineamiento exacto de secuencias de ADN**, o problema de búsqueda de patrones, determina las coincidencias exactas de cadenas de nucleótidos en una larga secuencia de ADN.

La tarea resuelve este problema para una secuencia de ADN principal aleatoria y múltiples patrones de nucleótidos que también pueden ser secuencias aleatorias y/o copias exactas de partes de la secuencia principal ubicadas aleatoriamente. Se elige un algoritmo simple de fuerza bruta que verifica cada patrón comenzando en cada posición posible de la secuencia de ADN.

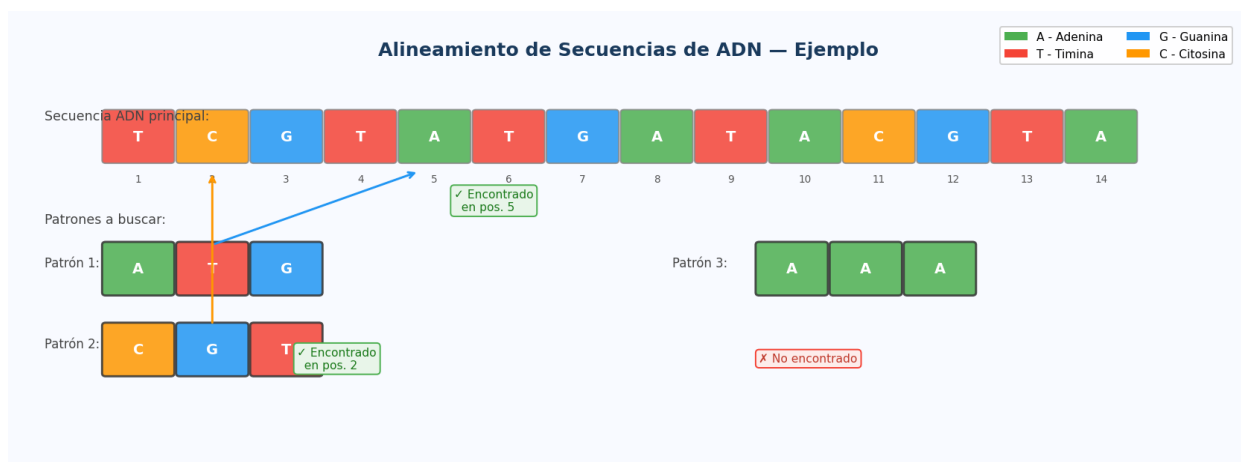


Figura 1 — Ejemplo de alineamiento de secuencias de ADN con múltiples patrones

II. Conceptos Cubiertos

Esta tarea cubre los siguientes conceptos principales de programación paralela:

- Paralelización de bucles con dos bucles anidados que pueden paralelizarse independientemente o conjuntamente.
- Detección y resolución de condiciones de carrera para escritura y actualización cuando se gestionan contadores de coincidencias.
- Soluciones con regiones críticas, operaciones atómicas o sustituciones de reducción.
- En MPI: distribución de la secuencia de ADN principal entre procesos, con pipeline y comunicaciones punto a punto.
- Generación paralela de secuencias pseudoaleatorias.
- Estrategias para notificar y detener cálculos especulativos cuando se encuentran coincidencias.

PARTE II: Especificación del Proyecto

Alineamiento de Secuencias de ADN

Implementación Paralela y Concurrente con Pthreads, MPI y OpenCL

Complejidad	Tecnologías	Lenguaje
Baja-Intermedia	Pthreads · MPI · OpenCL	C / C++

1. Introducción y Motivación

El ADN (ácido desoxirribonucleico) es la molécula que contiene la información genética de todos los seres vivos. Puede representarse como una cadena de caracteres formada por cuatro letras: **A** (Adenina), **T** (Timina), **G** (Guanina) y **C** (Citosina).

Buscar patrones dentro de secuencias de ADN es una tarea fundamental en bioinformática: permite identificar genes, detectar mutaciones y comparar organismos. Sin embargo, las secuencias reales pueden tener millones o incluso miles de millones de nucleótidos, por lo que buscar patrones de forma secuencial (uno por uno) puede tardar mucho tiempo.

En este proyecto aprenderán a acelerar esa búsqueda usando programación paralela: dividirán el trabajo entre varios hilos o procesos que trabajen al mismo tiempo, logrando que el programa sea mucho más rápido.

Analogía sencilla: Imaginen que tienen un libro muy largo y necesitan encontrar cuántas veces aparece una palabra. Si lo buscan solos, tardará mucho. Pero si 4 personas buscan cada una en una parte del libro al mismo tiempo, terminan 4 veces más rápido. Eso es exactamente lo que harán en este proyecto con el ADN.

2. Descripción del Problema

2.1 El Algoritmo de Fuerza Bruta

El algoritmo que implementarán funciona de la siguiente manera:

1. Se tiene una secuencia de ADN principal de longitud N (por ejemplo, 10 millones de nucleótidos).
2. Se tienen K patrones a buscar (por ejemplo, 100 patrones de longitud variable).
3. Para cada patrón, el programa prueba si coincide en cada posición posible de la secuencia principal.
4. Si el patrón coincide, se registra la primera posición donde ocurre y se detiene la búsqueda de ese patrón.
5. Al final, se reporta qué patrones fueron encontrados y en qué posición.

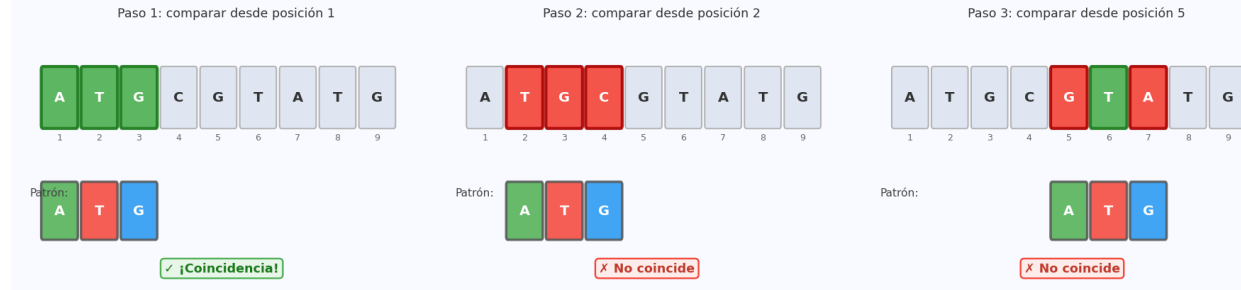
Algoritmo de Fuerza Bruta — Proceso de Comparación Paso a Paso

Figura 2 — Proceso paso a paso del algoritmo de fuerza bruta para la búsqueda de patrones

2.2 Ejemplo Concreto

Supongamos que la secuencia de ADN es:

```
secuencia = "TCGTATGATACGTA"
patron_1  = "ATG"    --> encontrado en posicion 5
patron_2  = "CGT"    --> encontrado en posicion 2
patron_3  = "AAA"    --> no encontrado
```

2.3 Pseudocódigo Secuencial

Para ayudarles a entender, aquí está el pseudocódigo de la solución secuencial que deben implementar primero:

```
// Fase 1: Generar secuencia de ADN aleatoria
generar_secuencia_adn(N)          // N nucleotidos
// Fase 2: Generar patrones aleatorios
generar_patrones(K, longitud_max)
// Fase 3: Búsqueda (parte a paralelizar)
PARA CADA patron p en [0..K-1]:
    encontrado = FALSO
    PARA CADA posicion i en [0..N-longitud_patron]:
        SI coincide(secuencia[i..], patron[p]):
            resultado[p] = i    // guardar posicion
            encontrado = VERDADERO
            ROMPER el bucle interior
// Fase 4: Reportar resultados
imprimir_resultados()
```

3. Cronograma del Proyecto



Figura 3 — Cronograma general del proyecto con sus tres fases de entrega

ENTREGABLES DEL PROYECTO

Entregable 1

Semana 8

Diseño e Implementación con Pthreads y MPI

Entregable 1 — Descripción General

En este primer entregable, el objetivo es **diseñar su solución completa** e implementar el algoritmo de búsqueda usando las dos primeras tecnologías de paralelismo: **Pthreads** (hilos en memoria compartida) y **MPI** (procesos en memoria distribuida).

IMPORTANTE: Antes de escribir una sola línea de código paralelo, deben tener muy claro el diseño de su solución. Un buen diseño previo ahorra muchas horas de depuración de errores.

Parte A — Implementación Secuencial Base

Antes de paralelizar, deben implementar y validar la versión secuencial completa. Esta versión es su referencia para verificar que las versiones paralelas producen resultados correctos.

Requisitos de la versión secuencial:

- Generar una secuencia de ADN aleatoria de longitud configurable por argumento (ejemplo: -n 1000000 para 1 millón de nucleótidos).
- Generar K patrones aleatorios de longitud configurable (ejemplo: -k 100 -l 10 para 100 patrones de longitud 10).
- Implementar la búsqueda por fuerza bruta como se describió en la sección 2.
- Al finalizar, imprimir: cantidad de patrones encontrados, sus posiciones, y el tiempo de ejecución.
- Verificar con al menos 5 casos de prueba que los resultados son correctos (ver sección de casos de prueba).

Parte B — Implementación con Pthreads

Pthreads es una biblioteca de C para crear y gestionar hilos. Los hilos comparten la memoria del proceso, por lo que pueden acceder a los mismos datos sin necesidad de enviar mensajes. El reto principal es evitar condiciones de carrera.



Figura 4 — División del trabajo entre hilos con Pthreads

Estrategia de paralelización sugerida:

- Dividir los patrones entre los hilos: si hay 100 patrones y 4 hilos, cada hilo busca 25 patrones.
- Cada hilo escribe su resultado en su propia posición del arreglo de resultados (sin conflicto).
- Usar `pthread_mutex` para proteger el contador total de coincidencias (condición de carrera).
- Usar `pthread_barrier` para sincronizar hilos al finalizar.

Código mínimo de ejemplo para crear un hilo con Pthreads:

```
#include <pthread.h>

void* funcion_hilo(void* arg) {
    int id = *((int*)arg);    // id del hilo
    // ... trabajo del hilo ...
    return NULL;
}

// En main:
pthread_t hilos[NUM_HILOS];
pthread_create(&hilos[i], NULL, funcion_hilo, &id);
pthread_join(hilos[i], NULL);    // esperar al hilo
```

Parte C — Implementación con MPI

MPI (Message Passing Interface) es una biblioteca que permite que múltiples procesos, que pueden estar en diferentes computadoras, trabajen juntos enviándose mensajes. Cada proceso tiene su propia memoria separada.

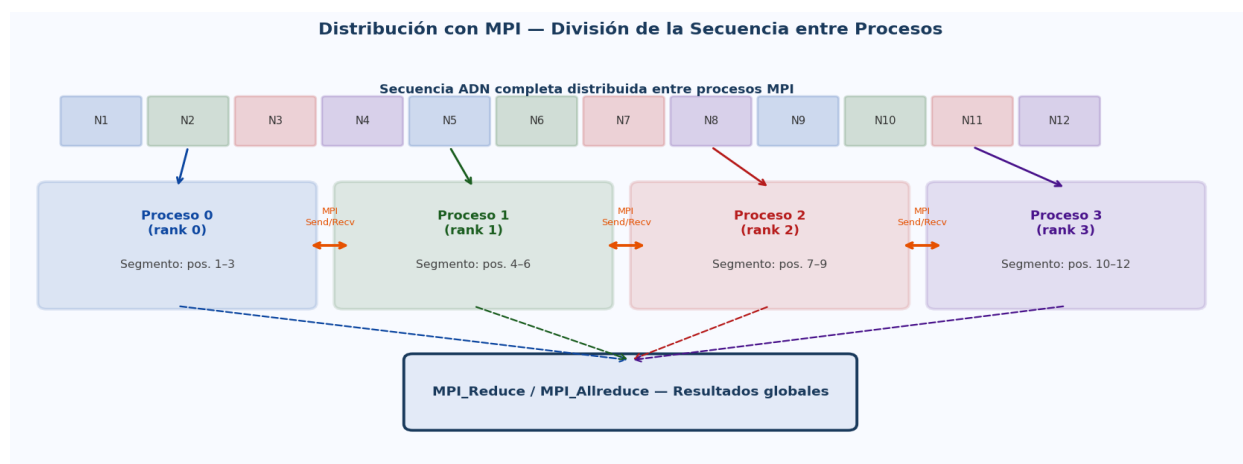


Figura 5 — Distribución de la secuencia de ADN entre procesos MPI

Estrategia de paralelización sugerida con MPI:

- El proceso 0 (root) genera la secuencia de ADN y todos los patrones.
- Usar MPI_Bcast para enviar la secuencia y los patrones a todos los procesos.
- Dividir los patrones entre los procesos (similar a la división de Pthreads).
- Cada proceso busca sus patrones asignados de forma independiente.
- Usar MPI_Reduce o MPI_Gather para recopilar todos los resultados en el proceso 0.
- El proceso 0 imprime los resultados finales.

Nota sobre MPI avanzado: Para quienes quieran ir más lejos, también pueden distribuir segmentos de la secuencia de ADN (no solo los patrones) entre los procesos, usando comunicaciones punto a punto (MPI_Send / MPI_Recv) cuando un patrón cruza el límite entre dos segmentos.

Parte D — Diseño del Sistema (Documento)

Junto con el código, deben entregar un documento corto (máximo 4 páginas) con:

1. Diagrama de su diseño: cómo dividieron el trabajo (un dibujo simple está bien).
2. Descripción de qué hace cada función o módulo principal.
3. Cómo evitaron las condiciones de carrera (qué mecanismos usaron y por qué).
4. Tabla con los resultados de sus casos de prueba.

Casos de Prueba — Entregable 1

Deben incluir al menos los siguientes casos de prueba en su entrega:

#	Secuencia ADN	Patrón	Resultado esperado	Descripción
1	"ATGCGT"	"ATG"	Posición 0	Patrón al inicio de la secuencia
2	"ATGCGT"	"CGT"	Posición 3	Patrón al final de la secuencia

3	"ATGCGT"	"AAA"	No encontrado	Patrón inexistente en la secuencia
4	"AAAAAA"	"AAA"	Posición 0	Patrón aparece múltiples veces (registrar solo la primera)
5	N=100,000	K=50 patrones	Mismo resultado seq. vs paralelo	Verificar que versión paralela produce resultados idénticos a la secuencial

Rúbrica — Entregable 1 (25 puntos)

Criterio	Puntos	Detalle
El programa compila y ejecuta sin errores (OBLIGATORIO)	5 pts	Sin esto, el resto no se califica. Compilar con gcc -o programa programa.c -lpthread
Implementación secuencial correcta	4 pts	Búsqueda correcta, resultados verificados con casos de prueba
Implementación con Pthreads	7 pts	División correcta del trabajo, uso adecuado de mutex, resultados idénticos a versión secuencial
Implementación con MPI	7 pts	Comunicaciones correctas con MPI_Bcast y MPI_Reduce, resultados correctos
Documento de diseño y casos de prueba	2 pts	Diagrama claro, descripción de mecanismos de sincronización, tabla de pruebas

Entregable 2

Semana 12

Optimización con Concurrencia y OpenCL

Entregable 2 — Descripción General

En este segundo entregable, el objetivo es **optimizar la solución del Entregable 1** incorporando técnicas avanzadas de concurrencia, y además implementar una versión usando **OpenCL** para ejecutar código en la GPU (o en la CPU con OpenCL si no hay GPU disponible). OpenCL es una alternativa abierta y compatible con cualquier fabricante de GPU.

¿Por qué OpenCL y no CUDA? CUDA solo funciona con tarjetas NVIDIA. OpenCL funciona con GPU de cualquier fabricante (AMD, Intel, NVIDIA) e incluso en la CPU. Por eso es la opción adecuada para este curso.

Parte A — Optimización de la Solución Pthreads

Deben mejorar la implementación de Pthreads del Entregable 1 con al menos dos de las siguientes optimizaciones:

- [object Object]: En lugar de crear y destruir hilos en cada búsqueda, mantener un conjunto de hilos siempre activos que reciban trabajo mediante una cola de tareas protegida con mutex.
- [object Object]: Asignar patrones a hilos dinámicamente (en lugar de estático) para evitar que algunos hilos terminen mucho antes que otros.
- [object Object]: Usar `pthread_cond_wait` y `pthread_cond_signal` para coordinar hilos de forma más eficiente en lugar de usar busy-waiting.
- [object Object]: Usar `pthread_rwlock` cuando múltiples hilos solo leen datos y solo uno escribe.

Parte B — Introducción a OpenCL

OpenCL permite escribir código (llamado "kernel") que se ejecuta en paralelo en miles de núcleos de la GPU. El modelo es similar a los hilos, pero a una escala mucho mayor.

Estructura básica de un programa OpenCL:

6. Obtener la plataforma y dispositivo disponible (GPU o CPU).
7. Crear un contexto y una cola de comandos.
8. Leer y compilar el kernel (código que corre en la GPU), escrito en OpenCL C.
9. Crear buffers de memoria en la GPU y copiar los datos (secuencia de ADN, patrones).
10. Ejecutar el kernel con tantos "work-items" como posiciones a revisar.
11. Leer los resultados de la GPU de vuelta a la CPU.

Esqueleto del kernel de OpenCL para la búsqueda:

```
// kernel.cl — Ejecuta en la GPU/CPU con OpenCL
__kernel void buscar_patron(
    __global const char* secuencia, // ADN en GPU
    __global const char* patron,    // patron a buscar
```

```

__global int* resultado,          // donde guardar pos.
int lon_patron, int lon_secuencia)
{
    // Cada work-item revisa UNA posicion de la secuencia
    int pos = get_global_id(0); // posicion de este work-item
    if (pos + lon_patron > lon_secuencia) return;
    for (int i = 0; i < lon_patron; i++) {
        if (secuencia[pos+i] != patron[i]) return;
    }
    resultado[0] = pos; // coincidencia encontrada
}

```

Parte C — Documento de Rediseño

Actualizar el documento de diseño del Entregable 1 con:

12. Descripción de las optimizaciones aplicadas a la versión Pthreads y por qué las eligieron.
13. Comparación de tiempos: versión secuencial vs Pthreads original vs Pthreads optimizado.
14. Explicación del diseño de la solución OpenCL.
15. Casos de prueba adicionales para validar las versiones optimizadas.

Rúbrica — Entregable 2 (35 puntos)

Criterio	Puntos	Detalle
El programa compila y ejecuta sin errores (OBLIGATORIO)	5 pts	Incluye la versión Pthreads optimizada y la versión OpenCL
Optimizaciones en Pthreads correctamente implementadas	10 pts	Al menos 2 optimizaciones, mejora medible en tiempo de ejecución
Implementación funcional de OpenCL	12 pts	Kernel correcto, manejo de buffers, resultados idénticos a versión secuencial
Comparación de rendimiento documentada	5 pts	Tabla o gráfica comparando tiempos de las tres versiones
Casos de prueba adicionales	3 pts	Pruebas para las versiones optimizadas

Entregable 3 Semana 17

Presentación Final y Análisis de Resultados

Entregable 3 — Descripción General

El tercer y último entregable consiste en una **presentación oral (10–15 minutos)** donde expondrán su solución completa y funcionando, junto con un análisis detallado de los resultados obtenidos. No es una exposición teórica: deben demostrar que su programa funciona y explicar qué aprendieron.

Estructura de la Presentación

16. Demostración en vivo (3 min): ejecutar el programa con una secuencia grande y mostrar los resultados.
17. Resultados de pruebas (2 min): mostrar la tabla de casos de prueba y confirmar que todas las versiones son correctas.
18. Análisis de rendimiento (5 min): gráficas de speedup y eficiencia, análisis con Ley de Amdahl.
19. Lecciones aprendidas (2 min): dificultades encontradas y cómo las resolvieron.
20. Preguntas del profesor (3 min): cada integrante del grupo debe poder responder.

La Ley de Amdahl — Qué es y cómo usarla

La Ley de Amdahl predice cuánto podemos acelerar un programa al paralelizar solo una parte de él. Es fundamental para entender los límites de la paralelización.

Ley de Amdahl: $\text{Speedup}(N) = 1 / [(1 - P) + P/N]$ donde: P = fracción del programa que puede paralelizarse, N = número de procesadores/hilos, $(1-P)$ = fracción que siempre es secuencial.

Ejemplo: Si el 90% del programa es paralelizable ($P=0.9$) y usamos 8 hilos ($N=8$):

```
Speedup = 1 / [(1 - 0.9) + 0.9/8]
         = 1 / [0.1 + 0.1125]
         = 1 / 0.2125
         = 4.7 veces mas rapido (no 8x como se esperaria)
```

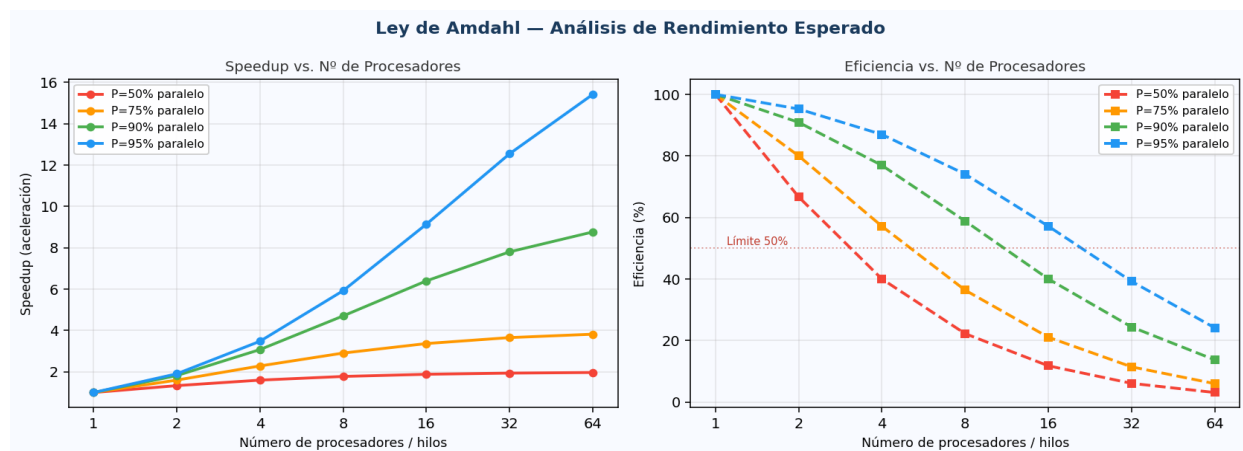


Figura 6 — Ley de Amdahl: Speedup y Eficiencia según fracción paralelizable

En su presentación deben:

- Medir el tiempo secuencial de su programa con al menos 3 tamaños de entrada diferentes.
- Medir el tiempo paralelo con 1, 2, 4 y 8 hilos/procesos (o los que tengan disponibles).
- Calcular el speedup real: $\text{Speedup_real} = T_{\text{secuencial}} / T_{\text{paralelo}}$.
- Estimar la fracción paralela P de su código (pueden medirla con herramientas de profiling).
- Comparar el speedup real con el predicho por la Ley de Amdahl y explicar las diferencias.

Tabla de Métricas a Presentar

Configuración	T. Secuencial (s)	T. Paralelo (s)	Speedup Real	Speedup Amdahl	Eficiencia (%)
Seq. (ref.)	—	—	—	—	—
Pthreads 2H	—	—	—	—	—
Pthreads 4H	—	—	—	—	—
Pthreads 8H	—	—	—	—	—
MPI 2P	—	—	—	—	—
MPI 4P	—	—	—	—	—
OpenCL	—	—	—	—	—

Rúbrica — Entregable 3 (40 puntos)

Criterio	Puntos	Detalle
Solución completa compila y ejecuta sin errores (OBLIGATORIO)	5 pts	Las 4 versiones: secuencial, Pthreads, MPI y OpenCL deben funcionar

Demostración en vivo funcional	8 pts	El programa produce resultados correctos con una secuencia grande en tiempo real
Presentación de resultados de pruebas	5 pts	Tabla completa con todos los casos de prueba, resultados verificados
Análisis con Ley de Amdahl	10 pts	Cálculo correcto, comparación con speedup real, discusión de diferencias
Gráficas de Speedup y Eficiencia	5 pts	Gráficas claras con al menos 4 configuraciones (1, 2, 4, 8 hilos/procesos)
Análisis de lecciones aprendidas	4 pts	Reflexión genuina sobre dificultades, condiciones de carrera, optimizaciones
Respuesta a preguntas (todos los integrantes)	3 pts	Cada persona del grupo demuestra entender su propio código

Puntos Extra — Interfaz Gráfica (Opcional)

Puntos extra: hasta 10 puntos adicionales sobre la nota final

Disenar e implementar una interfaz grafica para visualizar la busqueda de patrones en tiempo real.

La interfaz gráfica debe mostrar de forma visual cómo funciona la búsqueda paralela. No necesita ser compleja ni profesional. Se pueden usar bibliotecas como GTK, Qt, SDL2, o incluso una interfaz web sencilla. Lo importante es que sea funcional.

Requisitos Mínimos para Obtener los Puntos Extra

- Visualización de la secuencia de ADN (aunque sea los primeros 100 caracteres con colores por nucleótido).
- Visualización de los patrones a buscar y su estado: pendiente / encontrado / no encontrado.
- Indicación visual de cuántos hilos/procesos están activos y en qué parte de la secuencia trabajan.
- Botones para iniciar, pausar y reiniciar la búsqueda.
- Contador de coincidencias encontradas en tiempo real.

Ideas adicionales (para nota máxima)

- Animación del proceso de comparación: mostrar cómo el patrón se desliza sobre la secuencia.
- Gráfica de speedup que se actualiza automáticamente al cambiar el número de hilos.
- Modo de "pausa" que permite inspeccionar el estado de cada hilo.

Recursos y Referencias

Herramientas de Compilación

```
# Compilar version secuencial
gcc -O2 -o dna_sec dna_sec.c
# Compilar con Pthreads
gcc -O2 -o dna_pthread dna_pthread.c -lpthread
# Compilar con MPI
mpicc -O2 -o dna_mpi dna_mpi.c
# Ejecutar con MPI (4 procesos)
mpirun -np 4 ./dna_mpi -n 1000000 -k 100
# Compilar con OpenCL (Linux)
gcc -O2 -o dna_ocl dna_opencl.c -lOpenCL
```

Medir Tiempo de Ejecución en C

```
#include <time.h>
struct timespec inicio, fin;
clock_gettime(CLOCK_MONOTONIC, &inicio);
// ... codigo a medir ...
clock_gettime(CLOCK_MONOTONIC, &fin);
double tiempo = (fin.tv_sec - inicio.tv_sec) +
                (fin.tv_nsec - inicio.tv_nsec) / 1e9;
printf("Tiempo: %.4f segundos\n", tiempo);
```

Referencias Bibliográficas

- Gonzalez-Escribano, A., García-Álvarez, D., Cámara, J. "DNA sequence alignment: An assignment for OpenMP, MPI, and CUDA/OpenCL". EduHPC 2024. arXiv:2409.06075. Disponible en: <https://arxiv.org/abs/2409.06075>
- GAMUVa group. "Peachy Parallel Assignments". 2018-2024. Disponible en: <https://gamuva.infor.uva.es/peachy-assignments/>
- Barney, B. "POSIX Threads Programming". Lawrence Livermore National Laboratory. Disponible en: <https://hpc-tutorials.llnl.gov/posix/>
- Barney, B. "MPI Tutorial". Lawrence Livermore National Laboratory. Disponible en: <https://hpc-tutorials.llnl.gov/mpi/>
- Khronos Group. "OpenCL Programming Guide". Disponible en: <https://www.khronos.org/opencl/>
- OpenMPI. "Open MPI Documentation". Disponible en: <https://www.open-mpi.org/doc/>

Resumen de Ponderaciones del Proyecto

Entregable	Fecha	Puntos	% de la nota
------------	-------	--------	--------------

Entregable 1 — Diseño + Pthreads + MPI	Semana 8	25 pts	25%
Entregable 2 — Optimización + OpenCL	Semana 12	35 pts	35%
Entregable 3 — Presentación Final	Semana 17	40 pts	40%
Puntos extra — Interfaz gráfica (opcional)	Semana 17	+ hasta 10 pts	Adicional
TOTAL	—	100 pts	100%

Recuerden: en TODOS los entregables el programa debe compilar y ejecutar sin errores. Un programa que no compila recibe 0 en los criterios técnicos de ese entregable.

— Universidad de Costa Rica · Escuela de ECCT · Programación Paralela y Concurrente —